

CTF WRITEUP

Elephant in the Room

Written by Abdullah Sajid Malik

Author	Abdullah Sajid Malik
Category	Forensics
Difficulty	Easy
Flag	flag{M0wzUEg0TlQ1XzRSM19IVTYz}
Decoded	3L3PH4NT5_4R3_HU63

Challenge Description

"Sometimes the most important details aren't in the picture itself, but in how the file describes it. Look beyond the pixels. What does this elephant have to say for itself?"

We are given a single file:

We are given a single file: `stegpic.jpg` — an image of an elephant. The challenge nudges us away from the pixels and toward the file's metadata.

Reconnaissance

The first thing to do with any forensics challenge is look at what's actually in the file beyond what it renders as an image. JPEG files are structured with markers — special 2-byte sequences that delimit sections like image data, quantisation tables, and — crucially — comments.

The hint in the challenge description is very telling:

"Use a tool like ExifTool or an online metadata viewer to read the 'Comments' section of the file properties."

This tells us to focus on file metadata, not pixel-level steganography.

Solution

► Step 1 – Inspect the JPEG Structure

I scanned the raw bytes of stegpic.jpg for the JPEG Comment marker (0xFFFE). In the JPEG format, a comment block starts with:

```
FF FE → Comment marker
XX XX → Length of the comment (2 bytes, big-endian)
...   → Comment text (length - 2 bytes)
```

Running a scan of the file:

```
with open('stegpic.jpg', 'rb') as f:
    data = f.read()

idx = 0
while idx < len(data):
    pos = data.find(b'\xff\xfe', idx)
    if pos == -1:
        break
    length = int.from_bytes(data[pos+2:pos+4], 'big')
    comment = data[pos+4:pos+2+length]
    print(f'JPEG Comment at offset {pos}: {comment}')
    idx = pos + 1
```

Output:

```
JPEG Comment at offset 20: b'flag{M0wzUEg0TlQ1XzRSM19IVTYz}'
```

The flag is embedded directly in the JPEG comment field at byte offset 20 — practically at the very start of the file.

Note: This can also be found quickly with `strings stegpic.jpg | grep flag` or by opening the file in ExifTool and checking the Comment field.

► Step 2 – Decode the Flag Contents

The flag wrapper is `flag{...}` with the inner value `M0wzUEg0TlQ1XzRSM19IVTYz`. This looks like Base64 — the character set and the length are both consistent with it.

Decoding:

```
import base64
print(base64.b64decode('M0wzUEg0TlQ1XzRSM19IVTYz').decode())
```

Output:

```
3L3PH4NT5_4R3_HU63
```

Reading it phonetically: ELEPHANTS ARE HUGE — a clever reference to the challenge name "Elephant in the Room" and a leet-speak nod to the subject of the image. □

Flag

```
flag{M0wzUEg0TlQ1XzRSM19IVTYz}
```

Key Takeaways

- JPEG files contain more than pixels. The JPEG standard defines a comment marker (0xFFFE) that can store arbitrary text. Most image viewers never display this.
- Metadata is often overlooked. Tools like exiftool, strings, and hex editors reveal data that is invisible to a casual viewer — a classic forensics technique.
- Layered encoding is common in CTFs. The flag was wrapped in the standard flag{...} format and the contents were Base64-encoded. Always check if the inner value is encoded further.
- The file name and description are hints. "Elephant in the Room" is an idiom meaning something obvious that everyone ignores. The flag was sitting in plain sight inside the file.

Tools Used

Tool	Purpose
python3 (raw bytes)	Parse JPEG structure and find comment marker
strings	Quick grep for flag pattern
base64 (Python)	Decode the inner flag value
ExifTool (optional)	GUI/CLI alternative for metadata reading